Literature Review and Methodology

VulnLoc — An Explainable Hybrid LLM–GNN Framework for Line-Level Software Vulnerability Detection and Localization

> Draft sections for the paper. Written to be read as continuous prose. Citation numbers refer to the References list at the end. Numbers describing our own corpus come from measured runs in this repository, not from published papers.

---

2. Literature Review

2.1 Scope of the review

This review covers work published mainly between 2019 and 2026 on finding security vulnerabilities in source code using machine learning. We group the work by the question it answers rather than by date, because the field has moved along four separate lines at the same time: how code is represented, how fine-grained the prediction is, how different kinds of information are combined, and how trustworthy the labels and explanations are. We close by stating the gap that our work addresses.

2.2 Early deep learning and graph-based detection

The first wave of learning-based detectors treated a function as a sequence of tokens and trained a classifier to say whether it was vulnerable. These models were easy to build, but they ignored the fact that a program is not really a sequence. Two lines that sit far apart in the file can be closely related, and two lines that sit next to each other may have nothing to do with one another.

Graph-based methods were introduced to fix this. The key idea came from program analysis rather than machine learning. Yamaguchi et al. [1] proposed the **code property graph**, which merges three classic views of a program into one structure: the abstract syntax tree (AST), which records grammar; the control flow graph (CFG), which records the order in which statements can run; and program dependence information, which records how values move between statements. Their point was that many vulnerability patterns can only be described when all three views are available at once.

Devign [2] brought this idea into deep learning. It builds a composite graph for each function and runs a gated graph neural network over it. Devign showed that structure carries real signal, and it remains the standard reference point for structure-based detection. Its limitation is the one shared by nearly all of this generation of work: the output is a single label for the whole function.

2.3 Pre-trained language models for code

In parallel, the natural language processing field produced pre-trained transformer models for source code. CodeBERT [3] is trained on both natural language and programming language text, and produces general-purpose representations that can be fine-tuned for many code tasks. GraphCodeBERT [4] extends this by adding data flow information during pre-training, so the model learns where a variable's value comes from rather than treating the code as plain text.

These models raised reported accuracy considerably, and most recent vulnerability detectors use one of them as a starting point. However, several recent papers have questioned how much of that improvement is real. A 2025 survey of large language models in software security [19] reviews the field and lists interpretability and evaluation quality as unresolved problems. A critical audit of large language model based detection [20] argues that reported performance often does not survive careful evaluation. Another study re-evaluates pre-trained language model detectors under fairer conditions and finds smaller gains than originally claimed [21]. Recent work such as VULPO [22] continues to push what a language model alone can achieve by optimising it with better context, and is useful to us as a comparison point rather than as a competing method.

The overall message from this group is consistent and important for our design: pre-trained code models are strong, but they are not sufficient on their own.

2.4 Line-level and statement-level localization

The work closest to ours tries to move the prediction from the whole function to the individual line or statement.

LineVul [5] is the most widely cited example. It fine-tunes CodeBERT as a function-level classifier and then derives per-line scores from the model's attention weights. This was an important step, but the line scores are a by-product of a function-level model rather than something the model was trained to produce, and no program structure is used at all.

LineVD [6] approaches the same problem from the opposite direction. It treats statement-level detection as node classification on a graph, using control and data dependencies between statements. Here structure is central and the language model acts mainly as a source of node features. IVDetect [7] adds interpretability by using GNNExplainer [24] to isolate the part of the graph that most influenced the prediction.

Other recent approaches include learning a dictionary of vulnerability patterns and matching against it to locate statements [8], and a study showing that simply asking a large language model to check a program line by line already improves localization noticeably [9]. The latter result is instructive: it improves localization without using any program structure, which suggests that the structural side of the problem is still under-used.

Finally, a 2025 study in *Machine Learning* [10] evaluates how well existing detectors actually localize, as opposed to how well they classify. Its conclusion is that localization quality is generally poor and under-measured. We treat this paper as evidence that the problem is open, and we borrow parts of its evaluation protocol.

2.5 Combining semantic and structural information

Since both the text view and the graph view carry useful information, a natural step is to use both. Most existing systems do this by **concatenation**: each branch produces one summary vector for the function, the two vectors are joined end to end, and a classifier reads the result. The two branches never influence each other.

Several recent papers try to do better. Vul-LMGNNs [11] combines language models with graph neural networks through online knowledge distillation. A 2025 study in *Knowledge-Based Systems* [12] uses a two-part transformer over source code and assembly code to capture fine-grained features and locate vulnerable statements. DCVD [13] proposes dual-channel cross-modal fusion for detection and localization together, and is the closest published system to our own design. Work in *Information Fusion* [14] takes a different second modality altogether, using explanations written by a language model and combining them with code features through a modified self-attention mechanism. VulReaD [15] guides reasoning with an external knowledge graph, and other work injects a semantic vulnerability graph into transformer training [16].

A 2026 review of graph neural networks for vulnerability mining [23] surveys this area systematically and identifies graph and language model integration as a live research direction. Separately, work on causal deep learning for vulnerability detection [17] attacks a different weakness, namely that these models often rely on shortcut features that do not actually cause the vulnerability.

Two observations matter for our work. First, most of these systems still fuse at the level of the whole function, so any per-line detail is lost before the two views meet. Second, and more importantly, **none of them reports a controlled comparison between deep interaction and simple concatenation** under matched training conditions. Improvements are reported against older baselines, not against the same system with the fusion step replaced.

2.6 Explainability and faithfulness

Explanations are often listed as a contribution, but the standard of evidence varies widely. Many systems present attention weights as an explanation. Jain and Wallace [25] showed that this is unsafe: it is possible to construct completely different attention distributions that produce essentially the same prediction, which means the attended tokens do not reliably tell us what the model depended on.

The interpretability literature offers stronger tests. The ERASER benchmark [26] proposes measuring *comprehensiveness* and *sufficiency*: if the parts the model claims to rely on are removed, the prediction should change a great deal, and those parts alone should be enough to support the prediction. GNNExplainer [24] plays a similar role for graph models by finding the smallest subgraph that preserves the prediction.

In the vulnerability domain specifically, recent work instruction-tunes a large language model to produce written explanations of vulnerabilities [18]. The explanations read well, but there is no built-in guarantee that they describe what the model actually used. This is the risk we design against directly.

2.7 Data quality

A separate body of work studies the datasets themselves, and it is essential for anyone reporting line-level numbers.

Big-Vul [27] collects C and C++ functions from real projects, linked to published CVE records, and records which lines a fix commit changed. It is the main public source of line-level labels for C and C++, which is why we use it. CVEfixes [34] and MegaVul [32] provide related collections built from vulnerability-fixing commits.

The problem is that these labels are derived automatically, and a fix commit usually contains more than the fix. Croft et al. [28] measured label accuracy across several public datasets and reported between 20% and 71% inaccurate labels, along with high levels of duplication. CleanVul [30] independently estimated 40% to 75% noise and proposed a method to reduce it, and ActiveClean [33] tackles the same problem using active learning.

PrimeVul [29] goes further. It rebuilds an evaluation set with stricter labelling and de-duplication and shows that much of the reported progress in this field shrinks or disappears under a cleaner evaluation. DiverseVul [31] provides a large and CWE-rich collection that is useful for testing generalization, although it does not carry line labels.

The conclusion we take from this group is that **any line-level result reported without a label-quality estimate should be treated with caution**, including our own.

2.8 The research gap

Bringing these lines together, the literature establishes four things. Program structure carries signal that plain text does not [1, 2]. Pre-trained code models are powerful but not sufficient by themselves [3, 4, 20, 21]. Developers need line-level output, and current systems provide it poorly [5, 6, 10]. Labels in this domain are noisy enough to affect conclusions [28, 29, 30].

What is missing is the combination. No published work, as far as we can determine, lets the semantic and structural views **interact in both directions on a shared line index before either is pooled**, and then measures that choice against plain concatenation under identical data, identical encoders, and an identical tuning budget. The closest systems [11, 12, 13] either fuse at the function level, use a different second modality, or do not run the controlled comparison. That comparison is the contribution of this work, together with a line-level evaluation protocol that reports its own label-quality ceiling.

---

3. Methodology

3.1 Overview

Our system takes one C or C++ function as input and produces a risk score for every line in it, together with a short explanation for the highest-scoring lines. It has two branches. The **semantic branch** reads the code as text using a pre-trained code model. The **structural branch** reads the code as a graph using a graph neural network. Both branches are converted to the same unit of measurement — the source line — and are then allowed to attend to each other before any final decision is made.

The design follows one rule throughout: **the line index is the atomic unit, and it must never be lost.** Every stage of the pipeline is built to preserve it.

The full pipeline has eight stages:

1. Normalize the source text, removing the formatting artefact described in Section 3.5 without changing the number of lines.
2. Parse the function into a syntax tree.
3. Build the AST, CFG and DFG over a shared set of nodes; the CPG is their union.
4. Encode the code as text (semantic branch) and as a graph (structural branch).
5. Convert both to one vector per source line.
6. Fuse the two using bidirectional cross-attention.
7. Predict a score per line and a score per function, and train on both.
8. Generate an explanation that is required to cite real evidence.

3.2 Problem formulation

Let a function be a sequence of lines $L = (l_1, \ldots, l_n)$. The task is to learn a function that maps the source code to a vector of scores $(s_1, \ldots, s_n)$, where $s_i \in [0, 1]$ is the probability that line l_i contains the vulnerability. We also predict a single function-level score $y \in [0, 1]$.

This is a ranking problem more than a classification problem. In practice a developer looks at the highest-scoring lines first, so what matters is whether the true vulnerable line appears near the top of the list, not whether its score crosses some fixed threshold. Our choice of evaluation metrics in Section 3.13 follows from this.

3.3 Datasets

We use five corpora, each with one clearly defined role. **The roles never share data.**

ROLE	CORPUS	REASON
Training	Big-Vul [27]	The only public C/C++ set that provides per-line flaw indices
Clean evaluation	PrimeVul confirmed subset [29]	Manually verified labels; measures the effect of label noise
Generalization	DiverseVul [31]	Projects that do not appear in Big-Vul
Validation	PrimeVul paired [29]	The same function before and after the fix
Pipeline testing	Juliet 1.3.1 [35]	Exact ground truth; used only for unit tests and figures

Two points need to be stated clearly, because they are easy to get wrong. First, PrimeVul is built partly *from* Big-Vul and related sources, so it is **not** an independent generalization set; it measures label quality. DiverseVul is the generalization set. Second, Juliet is artificial code, so it is never used for a headline result.

Our copy of Big-Vul contains 188,636 functions, of which 10,900 (5.78%) are labelled vulnerable and 8,243 carry usable line-level labels. Across the whole corpus, 0.795% of source lines are marked vulnerable. This severe class imbalance drives several later design choices.

3.4 Label derivation and data splitting

Line labels in Big-Vul come from vulnerability-fixing commits: the lines that the fix removed or changed are treated as the vulnerable lines. We verified that this derivation is applied consistently across our copy of the corpus, finding no out-of-range line numbers, no mismatches between the recorded line text and the source, and no labels attached to non-vulnerable functions.

Consistency, however, is not correctness. We manually audited a random sample of 20 labelled functions and judged 6 of them to be wrong, giving an estimated line-label error rate of about 30% (95% confidence interval 14.5% to 51.9%). This figure is a **ceiling on every line-level metric we report**, and we report it alongside our results. It is a different and structurally larger quantity than the function-level noise rate, and the two are never combined into a single number.

We split the data **by project**, in an 80/10/10 ratio. All functions from a given project go entirely into one split. A random split would place near-identical functions in both training and test data, because the same function often appears many times in a project's history; the resulting score would measure memorisation rather than learning. The resulting split contains 148,983 training, 18,593 validation and 18,442 test functions, with further sets withheld for clean evaluation.

Assignment is made by **hashing the content of each function**, not by shuffling with a random seed. A seeded shuffle produces a different result if the input file order ever changes; a content hash does not. Re-running the split script therefore verifies the existing split rather than producing a new one, and any difference is treated as an error.

We measured leakage directly and found no shared projects between splits, no duplicate function hashes across splits, and no clean-evaluation function present in training. We also found that a project-wise split is not sufficient on its own: 426 functions were byte-identical across different projects, because open-source code is copied and vendored between repositories. These were withheld. Since only exact matches are detected, this number is a lower bound.

3.5 Preprocessing: a formatting artefact in the training corpus

The widely used release of Big-Vul distributed with LineVul [5] carries a defect that makes line-level results on it meaningless unless it is removed first, and we document it here because it is not specific to our work.

The release provides each function as a `processed_func` field. That field is assembled by stripping leading whitespace from every line **except** the flagged ones, which are pasted back verbatim from the `flaw_line` field and therefore keep their original indentation. The label is consequently written into the formatting. We measured the resulting shortcut with a single rule — *does this line begin with a space or a tab?* — used as a classifier. It predicts the line label at **precision 1.0000 with zero false positives on all six partitions of our split**, at recall between 0.83 and 0.99, a 38-fold lift over the base rate on the manually confirmed evaluation set.

A model given that text does not need to read the code. Our first full training run reported line-level precision 0.985 and F1 0.961. A line-label error rate of roughly 30% (Section 3.4) does not permit scores anywhere near that, so the result was impossible before it was suspicious. Its validation F1 was also flat from the first epoch onwards, because there was nothing left to learn after the first. We discarded that run. Both signals generalise as diagnostics: **a held-out score above a stated noise ceiling is a bug report rather than a result, and a learning curve that is flat from epoch zero is the more sensitive of the two alarms.**

The correction is to strip whitespace from both ends of every line, applied at corpus load time and again when a function is turned into an analysis unit. The transform is idempotent and preserves the number of lines, so the line index — the atomic unit of this work — is untouched. It also leaves the data split bit-identical: assignment is keyed on a function hash that already removes every whitespace character, so the split fingerprint is unchanged and no re-partitioning was required. Only the stored source text differs.

We re-ran the same detector-as-audit over the corrected corpus and it now fires on zero lines in every partition. That audit is retained as a standing check and is re-run before any line-level number is quoted; the code and the decision record are released with the artefact. **Every result in this paper is computed on normalized text.** We note that published line-level results on this release of Big-Vul which do not describe an equivalent normalization step may be measuring the same artefact.

3.6 Code representation

Parsing. We parse with Tree-sitter [36]. Conventional program analysis tools require code that compiles, but our corpus consists of isolated functions with no headers and no build system, so almost none of it compiles. Tree-sitter parses incomplete code and recovers from errors, which makes it the only practical option at this scale.

Grammar selection. Big-Vul records a language for each function, and 98.6% of entries say "C". We found that trusting this field is the worst available option. Measured across the corpus, the declared language gives a clean parse for only 37.01% of functions, parsing everything as C gives 35.74%, and parsing everything as C++ gives 88.24%. We therefore parse each function as C++ first and, if damage remains, also as C, keeping whichever tree has less damage. Ties are resolved in favour of C++, so the chosen grammar depends only on the source text and the procedure is reproducible. This policy yields a clean parse for **89.14%** of the corpus.

The functions that no grammar parses cleanly are 10.86% of all functions but contain **31.58%** of all flagged lines, because they are much longer than average. We report the larger figure, since the smaller one understates the effect on the labelled data.

Graph construction. From the parse tree we build three graphs over a single shared set of nodes:

- the **AST**, containing every syntactic element;
- the **CFG**, connecting statements in the order they can execute;
- the **DFG**, connecting the definition of a value to each of its uses.

The **CPG** is the union of these three edge sets over the same nodes. Because all four graphs share one node set, the mapping from node to line is identical for all of them, and selecting a graph type for an ablation is a matter of choosing which edge types to include rather than rebuilding the graph.

Our data flow analysis is **name-based**: it matches variables by identifier, and therefore does not model pointer aliasing or scoping precisely. We state this limitation explicitly and do not claim that the model performs full data flow analysis.

Every node stores the line it belongs to, derived from its byte offset through a line map. Across the evaluation set, 98.78% of labelled lines carry at least one AST node; corpus wide, 7.84% of flagged lines carry no node at all, and 84.8% of those lie in functions that no grammar parses cleanly. The structural branch cannot score those lines, which is a second ceiling, separate from label noise.

3.7 Semantic branch

The semantic branch uses a pre-trained code model — CodeBERT [3] or GraphCodeBERT [4] — to produce a contextual representation of each token.

These models accept at most 512 tokens. Long functions exceed this, and the usual response is to truncate. We measured the cost of truncation and found it unacceptable: **38.1% of all flagged lines**, and 44.5% of those in the evaluation set, fall outside the first 512 tokens. A line that the model never reads cannot be localized.

We therefore process long functions in **overlapping windows** and combine the results. This retains 100% of flagged lines. One practical consequence is that a function is no longer a single forward pass: functions in the evaluation set require 4.17 windows on average, and the longest requires 287. Batching is therefore done by window, not by function.

Tokenization is line-aware: each token records the byte range it came from, so it can later be attributed to a line.

3.8 Structural branch

The structural branch runs a graph neural network over the graph produced in Section 3.6. Each node is initialised with an embedding of its node type together with an embedding of the subtokens of its own source text, and the network updates each node using information from its neighbours.

We treat the choice of architecture as an experimental question rather than a design decision, and compare graph convolutional, graph attention and gated graph networks at depths of 2, 4 and 6.

Relations are typed, or the model is told they are not. The code property graph is a union of syntactic, control-flow and data-flow edges, and passing it to a network that treats every edge alike scores a graph in which "this statement is syntactically nested inside that one" and "a value flows from this statement to that one" are the same relation. Our encoder therefore learns one non-negative weight per relation and scales each edge's messages by it, using the same mechanism for all three architectures so that the comparison between them is not partly a comparison of edge handling. Treating a graph as untyped remains available, but only as an explicit setting: the default refuses a multi-relation graph rather than silently averaging it.

We measure what the node features can support before training anything. If two lines carry the same multiset of node features and one is flagged while the other is not, no model reading those features can separate them, whatever its architecture or training budget. We therefore compute that ambiguity directly, repeating it at increasing neighbourhood radius using Weisfeiler-Lehman refinement — the standard upper bound on what a message-passing network can distinguish. On our training split, the share of flagged lines that remain indistinguishable from an unflagged line is **76.2% using node types alone, 27.9% after two hops, 7.4% after four and 5.4% after six.**

This is not a diagnostic curiosity; it changes how the depth comparison may be read. A two-layer model carries a 27.9% ceiling on recall that comes from the representation rather than from the architecture, so its losing to a four-layer model is the expected outcome before over-smoothing is considered at all. **Only the comparison between six and four layers is informative about over-smoothing**, because there the two feature ceilings are nearly equal. We report the measurement alongside the sweep for that reason, and we recommend that any depth study on code graphs do the same: without it, a representational limit and an architectural one are indistinguishable in the results table.

Graph size varies enormously. AST graphs average 252 nodes across the corpus but average 1,144 nodes on the evaluation set and reach 56,266 nodes in the largest case. Since memory use in a batch is determined by the largest graph rather than the average, batching is sized against observed maxima — a batch is filled to a budget of nodes rather than of functions.

Two limits on this branch are specific to it and are reported with every structural result. **7.84% of flagged lines carry no graph node at all**, concentrated in the functions no grammar parses cleanly, so the branch cannot score them however well it is trained. And the subtoken vocabulary is built from the **training split only**: identifiers occurring exclusively in validation or test data are mapped to an unknown symbol rather than given their own embedding, because a vocabulary fitted over the evaluation data would leak it.

3.9 Line alignment

This module is the point on which the whole design depends. The semantic branch produces one vector per token; the structural branch produces one vector per graph node. These two sets of units do not correspond to each other, so the branches cannot be compared directly.

The only unit both agree on is the source line. The **LineAligner** therefore pools all token vectors belonging to line i into a single vector, and pools all node vectors belonging to line i into a single vector. The result is two matrices of the same shape: one vector per line from each branch. We compare mean, maximum and attention-based pooling experimentally.

Both poolings rely on the byte-to-line map established at parse time, which is why the line index is preserved so carefully at every earlier stage.

3.10 Cross-attention fusion

With both branches expressed per line, we apply **bidirectional cross-attention**. In one direction, the semantic representation of each line forms queries and the structural representations form keys and values, so each line's semantic view is updated using structural evidence. In the other direction, the roles are reversed. The two updated representations are then combined and passed on.

The intuition is straightforward. Evidence for a vulnerability is usually split between the two views. "This statement copies memory" is a semantic observation. "The value being copied came from an earlier statement without a size check" is a structural one. Neither is alarming alone; together they describe a buffer overflow. Concatenation asks a final classifier to notice such combinations from two compressed summaries. Cross-attention allows the combination to be formed per line, while the detail still exists.

We do not assume this works. Section 3.14 describes the experiment designed to test it.

3.11 Prediction heads and training objective

Two heads sit on top of the fused representation. The **line head** produces a score for each line. The **unit head** pools across lines and produces one score for the function. Training uses a weighted sum of a line-level loss and a function-level loss. The function-level term is included because function labels exist for far more data than line labels: 10,900 functions are labelled vulnerable, but only 8,243 carry line labels.

Because only 0.795% of lines are vulnerable, a model trained with plain binary cross-entropy can achieve very high accuracy by predicting "clean" everywhere. We therefore compare plain, class-weighted and focal losses, and we exclude accuracy from our headline metrics.

3.12 Explanation

For the highest-scoring lines the system produces a short explanation with a fixed structure: the vulnerability type, the line, a one-sentence reason, and the evidence.

The important constraint is that the explanation must be **grounded**. It may only cite line numbers and symbol names that appear in the code and were used by the model. If grounding cannot be established, no explanation is shown. This is a direct response to the finding that fluent explanations may not reflect the model's actual reasoning [18, 25].

3.13 Evaluation

We fixed our headline metrics before running experiments, to avoid selecting metrics after seeing results.

Line level (primary). Top-k accuracy for k = 1, 3, 5, 10, which asks whether a truly vulnerable line appears among the k highest-scoring lines. Initial False Alarms (IFA), the number of clean lines a developer inspects before reaching a true one. Line-level PR-AUC, which is threshold-free and meaningful when positives are under 1% of the data. We also report Recall@k%LOC, mean first rank and mean average rank, and line-level precision, recall and F1 at a calibrated threshold. The headline metrics are **Top-5, IFA and line-level PR-AUC**.

Function level. Precision, recall, F1, MCC, ROC-AUC, PR-AUC and false positive rate. Accuracy is reported for completeness but never used as a headline number.

Explanation quality. We report a grounding rate, which measures the proportion of explanations citing only lines and symbols that genuinely exist, and we plan a human evaluation covering correctness, faithfulness, usefulness and clarity, with inter-rater agreement reported.

Faithfulness. Following ERASER [26], we apply deletion and insertion tests: removing the lines the model claims to rely on should sharply reduce its confidence, while removing random lines should not. We also measure agreement between attention weights and gradient-based attribution. Attention is treated as a hypothesis to be tested, not as an explanation in itself [25].

3.14 Experimental design and ablations

The central experiment, which we label **A1**, compares five fusion strategies: cross-attention, concatenation, summation, gating, and no fusion. All five are run with identical data, identical encoders, an identical training budget, and — critically — an **identical hyperparameter search budget**. Giving the proposed method more tuning effort than the baseline would invalidate the comparison, so the budget is fixed and recorded in advance. Each configuration is run with five random seeds and reported as mean and standard deviation with a significance test. Summation and gating are included specifically to rule out the explanation that any fusion mechanism would help equally.

Further ablations cover branch removal (full, semantic only, structural only), graph representation (AST, CFG, DFG, AST+CFG, CPG), graph network architecture and depth, choice of pre-trained encoder and whether it is frozen, attention direction (bidirectional versus each single direction), line pooling method, loss composition, class-imbalance handling, and explanation strategy.

Every ablation is a configuration change rather than a code change, so no experiment can accidentally alter the system it is being compared against.

Reporting commitment. If cross-attention does not significantly outperform concatenation under these matched conditions, we will report that result. In that case the contribution becomes the controlled comparison itself, together with the evaluation protocol and the label-quality analysis.

3.15 Implementation

The system is implemented in Python 3.12 using PyTorch. The code is organised into layers — data, representation, models, training, evaluation, explanation and serving — with imports permitted only in one direction; this is checked automatically, so an incorrect dependency fails the build. Extension points are registries, so new graph builders, networks, fusion methods and explainers are added by configuration.

Experiments are configured with Hydra and tracked with MLflow, and every reported number is tied to a logged run identifier. Training runs on a single 4 GB GPU using mixed precision and gradient checkpointing, measured at a peak of 2,456 MiB with a batch of 16 windows.

3.16 Threats to validity

Internal. Label noise of approximately 30% at the line level limits every line-level result; we report it explicitly and use a manually confirmed evaluation set to separate model error from label error. A second, sharper threat is the formatting artefact of Section 3.5: on the uncorrected corpus a line-level score is dominated by indentation rather than by code, and the failure is silent — the model trains normally and reports near-perfect numbers. We removed it, verified the removal with a standing audit, and discarded the run that preceded the fix; we treat any held-out score above the stated noise ceiling as evidence of a defect rather than of performance. Unequal tuning effort between fusion strategies would invalidate our main claim, which is why the search budget is fixed in advance.

External. Our results apply to C and C++ functions analysed one at a time, with no cross-function analysis. Roughly 10.86% of functions do not parse cleanly, and these contain 31.58% of flagged lines. We test cross-dataset generalization on DiverseVul, because single-dataset results do not establish real-world capability.

Construct. Attention is not by itself an explanation [25], so we test faithfulness rather than assuming it. Automatic text metrics do not measure usefulness, so human evaluation is primary for explanation quality. A third construct risk is specific to the depth study: a model's depth changes both how much context a line's representation carries and how much message passing can smooth it, and those two effects push in opposite directions. We separate them by measuring the representational ceiling at each depth independently of training (Section 3.8), and we restrict over-smoothing claims to the comparison where that ceiling is approximately constant.

Conclusion. Single-seed results are unreliable, so all comparisons use at least three seeds and the main experiment uses five, with significance testing.

References

[1] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and Discovering Vulnerabilities with Code Property Graphs," *IEEE Symposium on Security and Privacy (S&P)*, 2014, pp. 590–604. https://doi.org/10.1109/SP.2014.44

[2] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," *NeurIPS*, 2019. https://arxiv.org/abs/1909.03496

[3] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," *Findings of EMNLP*, 2020. https://arxiv.org/abs/2002.08155

[4] D. Guo et al., "GraphCodeBERT: Pre-training Code Representations with Data Flow," *ICLR*, 2021. https://arxiv.org/abs/2009.08366

[5] M. Fu and C. Tantithamthavorn, "LineVul: A Transformer-based Line-Level Vulnerability Prediction," *MSR*, 2022. https://doi.org/10.1145/3524842.3528452

[6] D. Hin, A. Kan, H. Chen, and M. A. Babar, "LineVD: Statement-level Vulnerability Detection using Graph Neural Networks," *MSR*, 2022. https://arxiv.org/abs/2203.05181

[7] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability Detection with Fine-grained Interpretations," *ESEC/FSE*, 2021. https://arxiv.org/abs/2106.10478

[8] "Learning to Quantize Vulnerability Patterns and Match to Locate Statement-Level Vulnerabilities," arXiv preprint, 2023. https://arxiv.org/abs/2306.06109

[9] "If LLMs Would Just Look: Simple Line-by-line Checking Improves Vulnerability Localization," arXiv preprint, 2024. https://arxiv.org/abs/2410.15288

[10] "Evaluating line-level localization ability of learning-based code vulnerability detection models," *Machine Learning*, 2025. https://doi.org/10.1007/s10994-025-06902-1

[11] "Vul-LMGNNs: Fusing Language Models and Online-Distilled Graph Neural Networks for Code Vulnerability Detection," arXiv preprint, 2024. https://arxiv.org/abs/2404.14719

[12] "Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture," *Knowledge-Based Systems*, 2025. https://www.sciencedirect.com/science/article/abs/pii/S0950705125003880

[13] "DCVD: Dual-Channel Cross-Modal Fusion for Joint Vulnerability Detection and Localization," arXiv preprint, 2026. https://arxiv.org/abs/2605.11015

[14] "Enhancing vulnerability detection by fusing code semantic features with LLM-generated explanations," *Information Fusion*, 2025. https://www.sciencedirect.com/science/article/pii/S1566253525005238

[15] "VulReaD: Knowledge-Graph-guided Software Vulnerability Reasoning and Detection," arXiv preprint, 2026. https://arxiv.org/abs/2602.10787

[16] "An Unbiased Transformer Source Code Learning with Semantic Vulnerability Graph," arXiv preprint, 2023. https://arxiv.org/abs/2304.11072

[17] "Towards Causal Deep Learning for Vulnerability Detection," arXiv preprint, 2023. https://arxiv.org/abs/2310.07958

[18] "Towards Explainable Vulnerability Detection with Large Language Models," arXiv preprint, 2024. https://arxiv.org/abs/2406.09701

[19] "Large Language Models in Software Security: A Survey of Vulnerability Detection Techniques and Insights," arXiv preprint, 2025. https://arxiv.org/abs/2502.07049

[20] "Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask," arXiv preprint, 2025. https://arxiv.org/abs/2504.13474

[21] "Revisiting Pre-trained Language Models for Vulnerability Detection," arXiv preprint,

1. https://arxiv.org/abs/2507.16887

[22] "VULPO: Context-Aware Vulnerability Detection via On-Policy LLM Optimization," arXiv preprint, 2025. https://arxiv.org/abs/2511.11896

[23] "Graph Neural Networks for Software Vulnerability Mining: A Review," *Information*,

1. https://doi.org/10.3390/info17080730

[24] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," *NeurIPS*, 2019. https://arxiv.org/abs/1903.03894

[25] S. Jain and B. C. Wallace, "Attention is not Explanation," *NAACL-HLT*, 2019. https://arxiv.org/abs/1902.10186

[26] J. DeYoung et al., "ERASER: A Benchmark to Evaluate Rationalized NLP Models," *ACL*,

1. https://arxiv.org/abs/1911.03429

[27] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries," *MSR*, 2020. https://doi.org/10.1145/3379597.3387501

[28] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data Quality for Software Vulnerability Datasets," *ICSE*, 2023. https://arxiv.org/abs/2301.05456

[29] Y. Ding et al., "Vulnerability Detection with Code Language Models: How Far Are We?" *ICSE*, 2025. https://arxiv.org/abs/2403.18624

[30] "CleanVul: Automatic Function-Level Vulnerability Detection in Code Commits Using LLM -Based Noise Reduction," arXiv preprint, 2024. https://arxiv.org/abs/2411.17274

[31] Y. Chen et al., "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection," *RAID*, 2023. https://doi.org/10.1145/3607199.3607242

[32] C. Ni et al., "MegaVul: A C/C++ Vulnerability Dataset with Comprehensive Code Representations," *MSR*, 2024. https://doi.org/10.1145/3643991.3644886

[33] "ActiveClean: Generating Line-Level Vulnerability Data via Active Learning," arXiv preprint, 2023. https://arxiv.org/abs/2312.01588

[34] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software," *PROMISE*, 2021. https://doi.org/10.1145/3475960.3475985

[35] NIST Software Assurance Reference Dataset, "Juliet Test Suite for C/C++, Version 1.3.1." https://samate.nist.gov/SARD/

[36] M. Brunsfeld et al., "Tree-sitter: An incremental parsing system for programming tools." https://tree-sitter.github.io/tree-sitter/